2022 IEEE International Conference on
Systems, Man, and Cybernetics (SMC)

A Data-Centric Approach to Evaluate Requirements Engineering in Multidisciplinary Projects

Ali Salmani[1], Alireza Imani[1], Majid Bahrehvar[1], Linda Duffett-Leger[2], Mohammad Moshirpour[1]

[1]Department of Electrical and Software Engineering, Schulich School of Engineering, University of Calgary, Calgary, Canada
[2]Faculty of Nursing, University of Calgary, Calgary, Canada

{ali.salmani1, alireza.imani, majid.bahrehvar, mmoshirpour, linda.duffettleger}@ucalgary.ca

2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC) | 978-1-6654-5258-8 /22/$31.00 ©2022 CRWN | DOI: 10.1109/SMC53654.2022.9945270

Multidisciplinary teams are often a necessity for software projects as they provide the required expertise to effectively solve complex problems. However, efficient collaboration between teams with different disciplines is challenging due to several factors such as considering gaps in knowledge areas, establishing a development process, and understanding different requirements from various groups or stakeholders. The agile methodology, such as scrum, offers a powerful approach to managing the software development process effectively. As part of the agile methodology, some techniques and tools are used to manage requirements change, which is a common practice in multidisciplinary teams. This research aims to leverage process-mining techniques to analyze data from Jira and GitHub to analyze the efficacy of software development process, particularly in multidisciplinary teams. This approach is applied to a case study of a virtual healthcare intervention system to measure the team's productivity. The results indicate several deficiencies in the process with respect to requirements engineering task that cause loss of time and increase rework rates. Results indicate that there are some challenges in the development process that contribute to some deficiencies. The rework rate is high and the number of tasks that are intended to be completed is less than what was planned. These factors can contribute to the lengthening of the software development process. Most of these challenges can be addressed by improving the requirement engineering process in order to obtain the requirements and manage change requests more efficiently.

***Index Terms*—Multidisciplinary, Agile methodology, Requirements Engineering, Process Mining**

I. Introduction

In health care and social care, software plays a significant role, taking on important challenges that aim to improve patient care and reduce costs [1]. The successful and practical implementation of healthcare-focused software applications requires close collaboration among researchers and practitioners with different but complementary interests using a multidisciplinary approach. A study tested the moderating effect of team processes on the multidisciplinary innovation relationship in two independent samples of health care teams. As a result, multidisciplinarity is positively related to the quality of team innovation if teams have a mature and well-established processes [2]. However, multidisciplinary collaboration between different stakeholders such as healthcare professionals and software developers can be challenging due to the lack of shared domain knowledge. Ineffective communication is the first and most important challenge observed in the management of such teams [3], [4]. In [3] AI developers faced various communication challenges. Also, stakeholders' communication issues including developers' lack of domain knowledge are perceived as a cause of faulty systems. Another study found that healthcare decision-making was challenged by a lack of technical knowledge or domain experts [4].

Scrum is the most widely used agile methodology employed to address complex adaptive problems, to efficiently and creatively delivering products of the highest possible value [5]. Requirements engineering in scrum is an informally specified process that relies heavily on individual knowledge and skill [6]. Instead of being done during a closed phase at the beginning of the development process, requirements engineering happens iteratively during the development process [7] . [6].

Process mining can be used to analyze the progress of the software development process by leveraging data from sources such as GitHub and Jira [8]. The lessons learned from process mining can be used to effectively improve the software development process and ultimately enhance the final product.

This research aims to study the performance of a multidisciplinary team in the development of a virtual health intervention system; Robin's Nest. Stakeholders include healthcare professionals such as nursing practitioners and researchers, counselors, administrators who work closely with the development team to communicate the requirements and test the evolving product. We employed process mining techniques to leverage the project data from sources such as Jira and GitHub to analyze the current process employed by the team. We found that the main challenges were in the process of requirements engineering which impacted design decisions and release planning. Our research found most challenges were due to insufficient domain knowledge and poor communication throughout the process, which can be attributed to the multidisciplinary nature of the team. On the basis of the gained experience during the project as well as the literature review, we have discussed how the challenges have been identified and have provided recommendations to overcome the difficulties of the existing approach.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:01:17 UTC from IEEE Xplore. Restrictions apply.

The rest of the paper is organized as follows. Section II presents Background and Related Work, the current existing approach is presented in section III. Evaluation of the existing framework, enumeration of the challenges and a set of solutions to overcome the challenges are presented in section IV while concluding remarks and future work is presented in section V.

II. Background and Related Work

A. Agile in a Multidisciplinary Team

The agile methodology differs from other software development methods in that it does not have clear-cut steps [9]. In place of specific processes or tools, it's an iterative methodology that stresses a set of best practices that emphasize the importance of people and their relationships [10]. Scrum is the popular agile methodology that is capable of helping teams move and learn faster and is mostly adopted by software companies. An investigation of the utility of scrum methodology found that 56% of teams used the methodology because it visualized the tasks on a workflow and divided complicated tasks into user stories [11], while 83% used a hybrid scrum approach. The challenge of implementing organizational design culture is the same regardless of the situation [11].

Pure scrum may not meet all the needs in a company exactly; for instance, the length of the sprints can be modified, a new role can be included for taking on some specific responsibilities throughout the project and in general, all the processes can be customized [12]. Many studies have been conducted to design a framework to deliver higher quality software development in the least time by tailoring scrum. Sanches et al, found that direct communication between different roles in a scrum team caused constant interruptions in the development process [13]. To meet this challenge a new role was included to fulfill the communication needs between teams involved in a distributed scenario. Rodriguez et al [14], introduced a scrum-based training model to maximize students' performance by adding a new role called agile-coach that helps students to adapt and improve their use and reinforces the use of agile methodologies and the development of nontechnical skills. Another study conducted on the effectiveness of agile in the Video Game Development (VGD) process. Researchers examined the challenges faced by a multidisciplinary team in developing a game development project. This study found that VGD had more needs than what is being met by the current agile frameworks. Scrum and Kanban frameworks, as well as other agile frameworks, often need to be modified to account for the different pipelines and milestones in the stages of the VGD process [15].

It can be concluded that scrum can be effectively incorporated into all software development processes with different goals. Based on our work, some challenges were observed in Requirement Change Management (RCM), Usability Testing, and Acceptance Testing. In the following section, we investigate the literature regarding the challenges with the agile process and its solutions.

B. Process Mining

Process mining involves extracting information about process behavior from event logs and it is considered a data-oriented discipline [16]. For organizations reliant on information systems to conduct their daily operations, process mining can be of great value [17]. By analyzing the run-time event data collected by those systems, a process mining analysis identifies the sequence of activities and the collaborations of users in a business process. There are many studies on the application of process mining in software development [18], [19]. Rubin et al. [20] explains how software configuration management system information can be used to construct explicit software development process models, which are often implicit and chaotic. Based on their findings, Disco and the ProM tool can be used to obtain process models from real-world software projects[20].

In [21], Arias, Marques and Rojas conducted a systematic mapping study to obtain an overview of the state-of-the-art on how process mining has been applied with agile methodology. They also addressed the most commonly used techniques and tools such as Disco and ProM [21].

The method of process mining described by Van der Aalst [16] consists of three types: discovery, conformance, and enhancement. In the discovery process, logs (records of events) are analyzed with the objective of obtaining a model of the process. As part of conformance checking analysis, a process model (ideal process) is compared with an event log (real process) to evaluate if the information contained in the event log (traditional process) is different from the process model (ideal process). The objective of the enhancement analysis is to extend or improve an existing process model based on the actual (and used) information about the process. Additionally, process mining analysis can be carried out according to four different perspectives [21]: controlflow, organizational, case and performance. The control-flow perspective focuses on the sequence of activities. The organizational perspective identifies those who perform the activities and how they are interconnected. Case perspective allows developers to analyze the characteristics that identify each case of the process. Finally, the performance perspective addresses the timing of the activity within the process.

C. Managing requirements changes

One of the inevitable phenomena in agile software development is the requirements changes. According to a survey [22], requirement changes can be categorized into following 11 groups: defect fixing, missing requirements, functionality enhancement, product strategy, design improvement, scope reduction, redundant functionality, obsolete functionality, erroneous requirements, resolving conflicts, and clarifying requirements. In out multidisciplinary project, we captured and discussed requirement changes in sprint review meetings and reported as new user stories to the project's Jira board.

Several studies have been conducted to meet the Requirement Change Management(RCM) challenges [23]–[25]. [23],

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:01:17 UTC from IEEE Xplore. Restrictions apply.

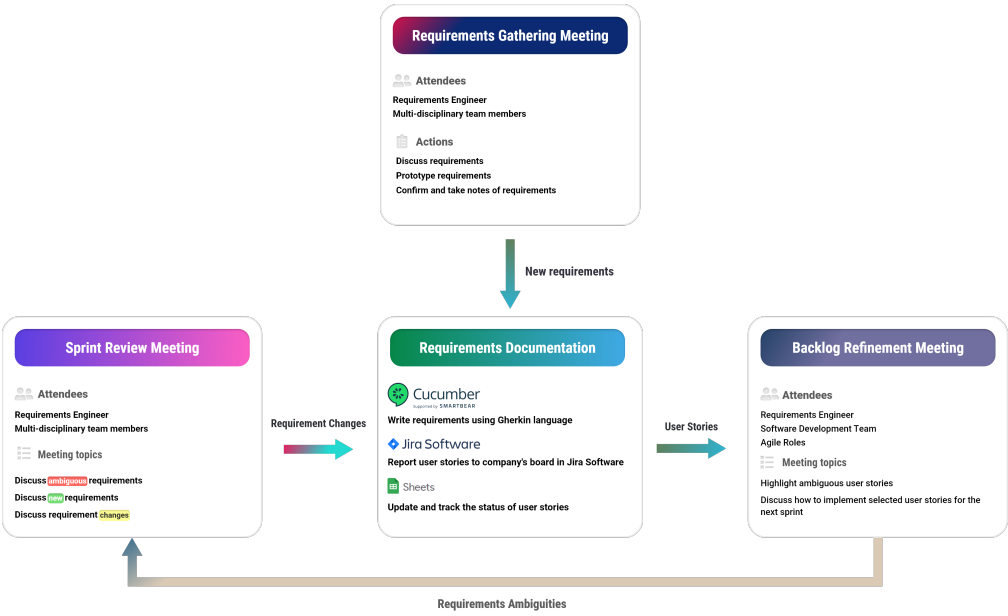

Fig. 1. Requirement Engineering Process

Shahzadi and her team conducted a framework to reduce the existing RCM gaps such as categorization problems, storage of implemented/unimplemented requirements, and stakeholder notifications. However, the framework may not be applicable for all teams with different circumstances. For instance, the requirement categorization prioritizes and measures the scope of required changes based on complexity and time costs [23], however, categorizing change requests are not be easy and straightforward in multidisciplinary teams with several different points of view.

Regarding requirements elicitation and product backlog initialization, Mahmud et al. [26] researched the impact of using the mind-mapping technique by stakeholders on elicitation and processing requirements in an agile development team. They evaluated the impact using an existing project as the case study. The outcome of their experiment is as follow:

- When mind mapping is adopted, the overall quality of the product backlog is significantly higher with "less expert" product owners and at least not worse with more expert product owners
- Mind-mapping and any derived technique should be considered as a valuable good practice to be pursued for setting up a promising initial product backlog

III. Case Study: Virtual Health Intervention Platform

We have presented a real-world virtual health intervention system, Robin's Nest, as a way of demonstrating our proposed software development approach. The Robin's Nest platform is used to deliver several different mental health programs/interventions and training courses virtually. The programs include VID-KIDS for new mothers with postpartum depression (PPD) and ATTACH for caregivers of children in domestic abuse situations who need mental health support. The development of Robin's Nest is a multidisciplinary project that involves a variety of different classes of stakeholders, including healthcare professionals, administrators, regulators, researchers, and patients. In light of accessibility issues and COVID-19 public health mandates, Robin's Nest provides an effective and efficient solution that leverages technology to help solve some of these challenges.

Managing the interactions among the team members during the development of a multidisciplinary software project can be challenging [27]. According to [27], RE is the process of eliciting the system requirements, devising solutions and designing decisions, and evaluating the solution based on the users' approval, also known as user acceptance testing. To evaluate the implemented software, another process called usability testing should also be performed. The successful implementation of eHealth apps relies heavily on usability, as with other digital applications [28]. By using this method, users and stakeholders can identify software defects. During prototyping phase, we also conduct usability testing so that changes can be made at a much lower cost.

Figure. 1 provides an overview of the software development process that was adopted by our team. During the requirement gathering process, domain experts from each nursing and engineering attended interdisciplinary meetings to discuss functional and non-functional requirements. Then the gathered requirements were converted into a semi-formally structured text using Gherkin [29]. In our software development process, we used behaviour-driven development (BDD), an agile approach to software development that encourages collaboration between developers, quality assurance testers, and customers. By establishing a shared understanding of how the application should behave using conversation and concrete examples,

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:01:17 UTC from IEEE Xplore. Restrictions apply.

teams can formalize their understanding [30]. Consequently, nurses reviewed the documents for validity and consistency.

Process evaluation relies on Jira and GitHub data. Control charts, velocity charts, and burn-down charts are provided in Jira reports showing team efficiency and progress. As an example, the burn-down chart shows how far the development team has progressed during each sprint. Data gathered from GitHub and Jira can provide further insight. Meta data such as commit data, creation data, and change status history can be used to determine the overall flow of the software process. This data can be used to answer questions such as: how many commits were performed per task? How long does it take for a task to be completed? Is the work status of the task changed? Is the task definition changed overtime?

The above questions will help us understand the workflow of each user story. In other words, we can monitor the tasks to see whether they've been clear enough for the developers or have changed over time throughout the process. In addition, the time to market of each requirement, and the rework rate of the team can be calculated. As the result, the efficiency and productivity of our team can be measured. We can also figure out what areas of our process can be improved, or we can deviate from it.

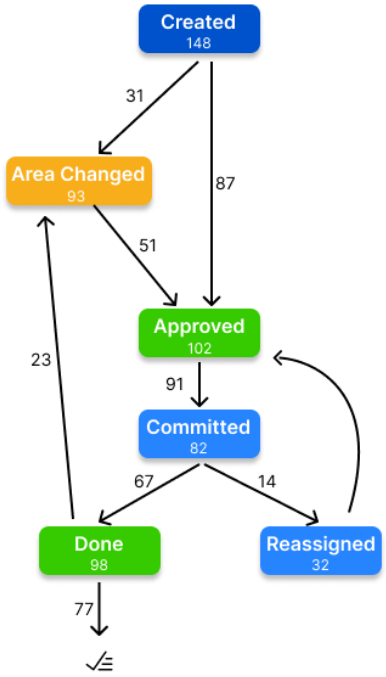

Fig. 2. Flow For Task Execution

IV. Process Evaluation

Process mining techniques were used to evaluate the efficiency of our software development process by analyzing a large amount of data extracted from GitHub and Jira. We considered the flow of data available for each requirement from the time it is entered into the process until it was implemented. After each task was created, the change-logs were carefully examined to determine: stage of the task after its creation, the number of tasks that were changed or approved, and the number of tasks that had been returned to the backlog due to requirement changes. Our evaluation found several areas where our processes could be improved to enhance the efficiency and effectiveness of our multidisciplinary team.

Figure 2 represents the flow to execute Product Backlog Items for tasks added in the backlog. There were a significant number of the items that had been changed at the "Done" state. Due to the considerable number of the "Area Changed" items, we can conclude that the requirements were often changed before being approved and even after being implemented.

According to the data that was extracted after analysing the process, most of the requirements were not implemented as expected. Therefore, more details were examined to calculate how long it would take to meet the requirements. Time-to-market [31] is a metric that is used in industry to measure the time it takes from conception to market release for a product. Figure 3, illustrates some of the topics that were discussed more frequently in the design sessions and the functional requirements required more details. For instance, as part of the VID-KIDS application, nurses should be able to enter the video call into the video log and have it appeared for playback during sessions with parents. Due to the changing requirements, six user stories had to be created in order to satisfy the requirement and reach the approval stage in four sprints. It likely caused by ineffective communication between multidisciplinary teams, including during requirements engineering meetings.

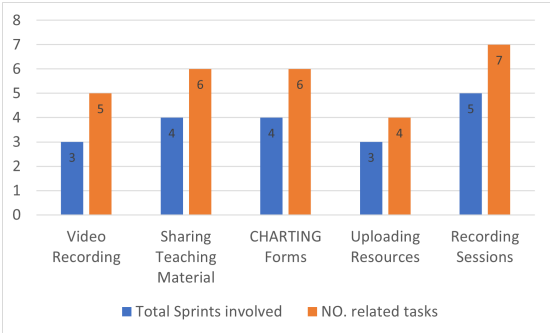

Fig. 3. Time taken for tasks that required several changes

Additionally, in some cases, the software requirements were altered during each sprint, which meant that the programmers had to modify the work they had previously completed on that sprint. The more time programmers spent on the requested changes for a specific user story, the less time they had to work on new user stories. As a result, ineffective communication caused a lot of rework and impeded software development. Figure 4 represents the progress of the project in one sprint. As it can be seen, 10 out of 16 tasks were completed throughout the sprint, and the rest are moved to the new sprint.

To calculate the rework rate, we used data automatically retrieved from Jira. The rework rate, defined as the ratio of rework hours to the productive labor hours, is calculated by

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:01:17 UTC from IEEE Xplore. Restrictions apply.

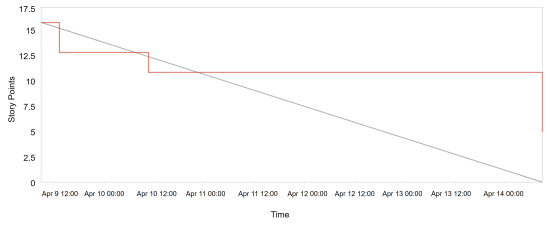

Fig. 4. The number of user stories remaining in a sprint

accumulating the number of hours dedicated on the requirement that has been implemented already. Figure 5 presents the monthly rework rates for five topics with the highest rework rate: The higher the value of the vertical axis, the higher the rework rate. For instance, the rework rate of the *Video Recording* requirement was 0.1 in August, 0.05 in September, and 0.05 in October, and 0.2 overall in the following three months.

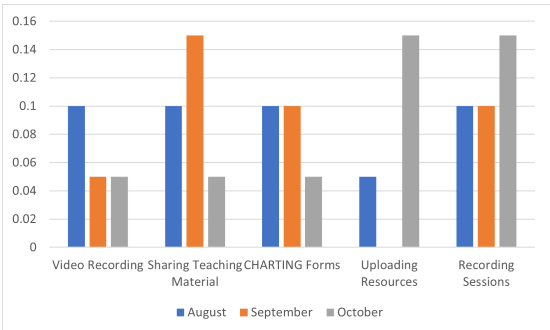

Fig. 5. Rework Rate

V. Diagnostics, Conclusions and Future Work

A. Diagnostics

By analyzing the data presented in the previous section, we have identified several areas of improvement in our process:

As mentioned, regular weekly meetings were scheduled to communicate with the stakeholders and to report the development team's progress, get the new requirements and make design decisions. Meeting attendees included representatives from the software development team and the nursing team who were involved in the project. However, some meetings were not always be attended by the same individuals which caused delays in decision making and ultimately resulted in a higher rework rate.

Further, considering the sensitive nature of e-health applications, the software developers may not take these aspects into consideration. As a result, getting the approval for new features was time-consuming. As a result, the development team was waiting for the new task to be finalized and assigned. According to Figure 2, a considerable amount of tasks were changed without adequate approval in an effort to meet development timelines. In addition, meeting attendees did not have a mutual understanding of one another. As a result, the team was unable to explore different solutions within the agreed timeframe and additional meeting time was often required for everyone to understand the problems and proposed solutions. This could cause the solution area to be changed after the user story was entered into the backlog.

In addition, although the software engineers who attended the requirements sessions had extensive experience as programmers, they were unaware of the nurses' concerns about the mothers' mental health and adding burden by soft launching the platform, which led nurses to place emphasis on the factors influencing patient satisfaction.

B. Conclusions and Future Work

This paper highlights the challenges of a multidisciplinary team by leveraging process mining techniques to evaluate the software development process. Several deficiencies were identified in the software development process, which negatively impacted the software development life cycle, including: high rework rates, a lack of mutual understanding between the team members, and occasionally changing requirements. Based on our findings one of the most important areas that could have been improved within our software development process was our approach to requirements engineering. It is now our intent to adopt some strategies in order to improve team performance in future projects. We strive to automate the requirement engineering process so that it is also user-centric in order to improve healthcare applications' user-friendliness. Also, for better data collection and analysis, we will use process mining tools such as ProM and Disco as our future work.

References

[1] J. H. Weber-Jahnke, M. Price, and J. Williams, "Software engineering in health care: Is it really different? and how to gain impact," *2013 5th International Workshop on Software Engineering in Health Care, SEHC 2013 - Proceedings*, pp. 1–4, 2013. DOI: 10.1109/SEHC.2013.6602469.

[2] D. Fay, C. Borrill, Z. Amir, R. Haward, and M. A. West, "Getting the most out of multidisciplinary teams: A multi-sample study of team innovation in health care," *Journal of Occupational and Organizational Psychology*, vol. 79, no. 4, pp. 553–567, Dec. 2006, ISSN: 2044-8325. DOI: 10.1348/096317905X72128. [Online]. Available: https://bpspsychub.onlinelibrary.wiley.com/doi/full/10.1348/096317905X72128%20https://bpspsychub.onlinelibrary.wiley.com/doi/abs/10.1348/096317905X72128%20https://bpspsychub.onlinelibrary.wiley.com/doi/10.1348/096317905X72128.

[3] D. Piorkowski, S. Park, A. Y. Wang, D. Wang, M. Muller, and F. Portnoy, "How AI Developers Overcome Communication Challenges in a Multidisciplinary Team: A Case Study," *Proc. ACM Hum.-Comput. Interact.*, vol. 5, no. CSCW1, Apr. 2021. DOI: 10.1145/3449205. [Online]. Available: https://doi.org/10.1145/3449205.

[4] B. Maqbool and S. Herold, "Challenges in Developing Software for the Swedish Healthcare Sector," [Online]. Available: https://drive.google.com/file/d/.

[5] *Scrum - what it is, how it works, and why it's awesome*. [Online]. Available: https://www.atlassian.com/agile/scrum.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:01:17 UTC from IEEE Xplore. Restrictions apply.

[6] A. Alhazmi and S. Huang, "Survey on Differences of Requirements Engineering for Traditional and Agile Development Processes," *Conference Proceedings - IEEE SOUTHEASTCON*, vol. 2020-January, 2020, ISSN: 07347502. DOI: 10.1109/SOUTHEASTCON44009.2020.9397492.

[7] E. M. Schön, J. Thomaschewski, and M. J. Escalona, "Agile Requirements Engineering: A systematic literature review," *Computer Standards & Interfaces*, vol. 49, pp. 79–91, Jan. 2017, ISSN: 0920-5489. DOI: 10.1016/J.CSI.2016.08.011.

[8] R. Marques, M. M. Da Silva, and D. R. Ferreira, "Assessing agile software development processes with process mining: A case study," *Proceeding - 2018 20th IEEE International Conference on Business Informatics, CBI 2018*, vol. 1, pp. 109–118, Aug. 2018. DOI: 10.1109/CBI.2018.00021.

[9] M. Jovanovic, A. Mesquida, N. Radaković, and A. Mas, "Agile retrospective games for different team development phases," *Journal of Universal Computer Science*, vol. 22, pp. 1489–1508, Apr. 2016. DOI: 10.3217/jucs-022-12-1489.

[10] K. Beck, M. Beedle, A. Van Bennekum, *et al.*, "Manifesto for Agile Software Development Twelve Principles of Agile Software," 2001. [Online]. Available: http://www.agilemanifesto.org.

[11] *Latest Reports on the Most Popular Agile Methodologies — Adeva*. [Online]. Available: https://adevait.com/blog/remote-work/adopting-agile-the-latest-reports-about-the-popular-mindset.

[12] *Agile Scrum - Tailoring Scrum to Your Needs*. [Online]. Available: https://smartbear.com/blog/agile-scrum-tailoring-scrum-to-your-needs/.

[13] *A SCRUM-Based Process to Distributed Projects in Multidisciplinary Teams: A Case Study — IEEE Conference Publication — IEEE Xplore*. [Online]. Available: https://ieeexplore.ieee.org/document/8560649.

[14] G. Rodriguez, A. Soria, and M. Campo, "Measuring the Impact of Agile Coaching on Students' Performance," *IEEE Transactions on Education*, vol. 59, no. 3, pp. 202–209, Aug. 2016. DOI: 10.1109/TE.2015.2506624.

[15] T. McKenzie, M. Morales-Trujillo, S. Lukosch, and S. Hoermann, "Is Agile Not Agile Enough? A Study on How Agile is Applied and Misapplied in the Video Game Development Industry," *Proceedings - 2021 IEEE/ACM Joint 15th International Conference on Software and System Processes and 16th ACM/IEEE International Conference on Global Software Engineering, ICSSP/ICGSE 2021*, pp. 94–105, May 2021. DOI: 10.1109/ICSSP-ICGSE52873.2021.00019.

[16] W. Van der Aalst, "Process mining: Data science in action," *Process Mining: Data Science in Action*, pp. 1–467, Jan. 2016. DOI: 10.1007/978-3-662-49851-4.

[17] H. Ma, "Process-aware information systems: Bridging people and software through process technology," *JASIST*, vol. 58, pp. 455–456, Apr. 2007. DOI: 10.1002/asi.20456.

[18] W. Poncin, A. Serebrenik, and M. Brand, "Process Mining Software Repositories," in *Proceedings of the European Conference on Software Maintenance and Reengineering, CSMR*, Apr. 2011, pp. 5–14. DOI: 10.1109/CSMR.2011.5.

[19] S. Bala, A. Solti, C. Cabanillas, J. Mendling, and A. Polleres, "Mining Project-Oriented Business Processes," vol. 9253, Apr. 2015, ISBN: 978-3-319-23062-7. DOI: 10.1007/978-3-319-23063-4{_}28.

[20] V. Rubin, C. Günther, W. Aalst, E. Kindler, B. Dongen, and W. Schäfer, "Process Mining Framework for Software Processes," Apr. 2007, pp. 169–181, ISBN: 978-3-540-72425-4. DOI: 10.1007/978-3-540-72426-1{_}15.

[21] M. Arias, M. R. Marques, and E. Rojas, "Using process mining in agile software development methodologies: A systematic mapping study," *Proceedings - 2018 44th Latin American Computing Conference, CLEI 2018*, pp. 552–561, Oct. 2018. DOI: 10.1109/CLEI.2018.00072.

[22] A. M. Alsalemi and E. T. Yeoh, "A survey on product backlog change management and requirement traceability in agile (Scrum)," *2015 9th Malaysian Software Engineering Conference, MySEC 2015*, pp. 189–194, May 2016. DOI: 10.1109/MYSEC.2015.7475219.

[23] Z. Shehzadi, F. Azam, M. W. Anwar, and I. Qasim, "A novel framework for change requirement management (CRM) in agile software development (ASD)," *ACM International Conference Proceeding Series*, pp. 22–26, Aug. 2019. DOI: 10.1145/3357419.3357438.

[24] S. Qureshi, S. U. R. Khan, Inayat-Ur-Rehman, Y. Javed, S. Saleem, and A. Iqbal, "A Conceptual Model to Address the Communication and Coordination Challenges during Requirements Change Management in Global Software Development," *IEEE Access*, vol. 9, pp. 102 290–102 303, 2021, ISSN: 21693536. DOI: 10.1109/ACCESS.2021.3091603.

[25] P. Marnada, T. Raharjo, B. Hardian, and A. Prasetyo, "Agile project management challenge in handling scope and change: A systematic literature review," *Procedia Computer Science*, vol. 197, pp. 290–300, Jan. 2022, ISSN: 1877-0509. DOI: 10.1016/J.PROCS.2021.12.143.

[26] I. Mahmud and V. Veneziano, "Mind-mapping: An effective technique to facilitate requirements engineering in agile software development," *14th International Conference on Computer and Information Technology, ICCIT 2011*, pp. 157–162, 2011. DOI: 10.1109/ICCITECHN.2011.6164775.

[27] K. L. Tan and W. B. Goh, "Designing a multi-disciplinary group project for computer science and engineering students," *IEEE Global Engineering Education Conference, EDUCON*, vol. April-2019, pp. 51–57, Apr. 2019. DOI: 10.1109/EDUCON.2019.8725147.

[28] I. Maramba, A. Chatterjee, and C. Newman, "Methods of usability testing in the development of eHealth applications: A scoping review," *International Journal of Medical Informatics*, vol. 126, pp. 95–104, Jun. 2019, ISSN: 1386-5056. DOI: 10.1016/J.IJMEDINF.2019.03.018.

[29] *BDD Testing & Collaboration Tools for Teams — Cucumber*. [Online]. Available: https://cucumber.io/.

[30] *Behavior-driven-development - Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Behavior-driven-development.

[31] *Time to Market: What it is & Why it's Important — TCGen*. [Online]. Available: https://www.tcgen.com/time-to-market/.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:01:17 UTC from IEEE Xplore. Restrictions apply.